Assignment: Implementing Symbol Table Tracking for Macro Definitions

Objective

The objective of this assignment is to implement a symbol table that tracks macro definitions and macro expansions in a compiler. The implementation ensures that macro names do not collide with ordinary identifiers and that macro expansion follows lexical scoping rules.

Introduction

Macros are compile-time entities that perform code substitution before semantic analysis. If macros are not managed correctly, they may conflict with variable names or expand incorrectly across scopes. Therefore, a compiler must maintain a symbol table that separates macros from ordinary identifiers and controls their visibility using lexical scoping.

Implementation Overview

The implementation uses a scoped symbol table where each scope maintains:

- A table for variables
- A table for macros

A stack of scopes is used to model lexical scoping. Macros and variables are checked for name collisions within the same scope, and macro expansion searches from the innermost scope outward.

Code Implementation

```
class SymbolTable:
    def __init__(self):
        self.scopes = []

    def enter_scope(self):
        self.scopes.append({"vars": {}, "macros": {}})

    def exit_scope(self):
        self.scopes.pop()

    def define_variable(self, name, value):
        if name in self.scopes[-1]["macros"]:
            raise Exception(f"Collision: '{name}' is a macro.")
        self.scopes[-1]["vars"][name] = value

    def define_macro(self, name, body):
        if name in self.scopes[-1]["vars"]:
            raise Exception(f"Collision: '{name}' is a variable.")
        self.scopes[-1]["macros"][name] = body

    def expand_macro(self, name):
        for scope in reversed(self.scopes):
            if name in scope["macros"]:
                return scope["macros"][name]
        raise Exception(f"Undefined macro: {na )
```

Example Usage

```
symbol_table = SymbolTable()
symbol_table.enter_scope()

symbol_table.define_macro("MAX", "(a > b ? a : b)")
print(symbol_table.expand_macro("MAX"))

symbol_table.define_variable("x", 10)

symbol_table.enter_scope()
symbol_table.define_macro("MIN", "(a < b ? a : b)")
print(symbol_table.expand_macro("MIN"))

symbol_table.exit_scope()
```

Explanation

- **Symbol Table**: Maintains separate namespaces for macros and variables.
- **Scope Management**: Uses a stack to ensure macros are valid only within their defining scope.
- **Collision Detection**: Prevents macro names from conflicting with variable names.
- **Macro Expansion**: Searches scopes from inner to outer to find the correct macro definition.

Conclusion

This assignment demonstrates a simple and effective approach to symbol table tracking for macro definitions. By separating macros from ordinary identifiers and enforcing lexical scoping,

the implementation prevents naming conflicts and ensures correct macro expansion during compilation.